

1. Minimizing lateness problem.

- Single resource processes one job at a time.
- Job i requires t_i units of processing time and is due at time d_i .
- If i starts at time s_i , it finishes at time $f_i = s_i + t_i$.
- Lateness: $\ell_i = \max\{0, f_i - d_i\}$.
- Goal: schedule all jobs to minimize maximum lateness $L = \max\{\ell_i \mid 1 \leq i \leq n\}$.

i	1	2	3	4	5	6	7
t_i	3	3	5	1	2	3	2
d_i	7	8	9	8	9	14	16

(1) 貪婪演算法, d_i 愈早, i 項愈早排

i	1	2	3	4	5	6	7
t_i	3	3	5	1	2	3	2
s_i	0	3	7	6	12	14	17
f_i	3	6	12	7	14	17	19
d_i	7 _①	8 _②	9 _④	8 _③	9 _⑤	14 _⑥	16 _⑦
ℓ_i	0	0	3	0	5	3	3

5分 $\rightarrow L = \max\{0, 3, 5\} = 5$

(2) d_i 由愈早, job 愈早執行

Algorithm:

① d_i 由小排到大, t_i 根據 d_i 結果一起排

② $s[i] = 0, f[i] = 0$

③ for 1 to N assume there are N jobs.

$$s[i] = f[i]$$

$$f[i] = s[i] + t[i]$$

$$l[i] = \max\{0, f[i] - d[i]\}$$

$$④ L = \max\{l, l + n\}$$

5分

如何
排程

找
lateness

i	1	2	3	4	5	6	7
t_i	3	3	5	1	2	3	2
d_i	7	8	9	8	9	14	16

1. 根據此表找最佳解

Greedy Algorithm $\rightarrow$ 截止時間(d_i)越早起先做!

Sort job by d_i and re number the job.

$\bar{i}$	1	2	3	4	5	6	7
t_i	3	3	1	5	2	3	2
d_i	7	8	8	9	9	14	16
s_i	0	3	6	7	12	14	17
f_i	3	6	7	12	14	17	19
l_i	0	0	0	3	5	3	3

$$L = \max\{0, 3, 5\} = 5$$

2. Design a polynomial-time Algorithm to solve this problem

Assume jobs struct exist job $1 \sim n$

MinLate(jobs, n):

sort jobs by d_i , 由小到大, 並重新編號 $\rightarrow O(n \lg n)$

jobs[0].s $\leftarrow 0$

jobs[0].f $\leftarrow 0$

$L \leftarrow 0$

for $\bar{i} \leftarrow 1 \sim n$:

jobs[$\bar{i}$].s = jobs[$\bar{i}-1$].f

jobs[$\bar{i}$].f = jobs[$\bar{i}$].s + jobs[$\bar{i}$].t

nowlate = jobs[$\bar{i}$].f - jobs[$\bar{i}$].d

if nowlate > 0:

$L = \max(\text{nowlate}, L)$

return L

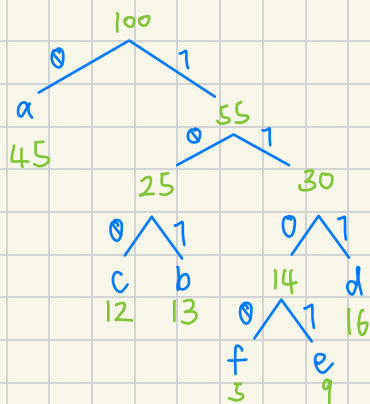
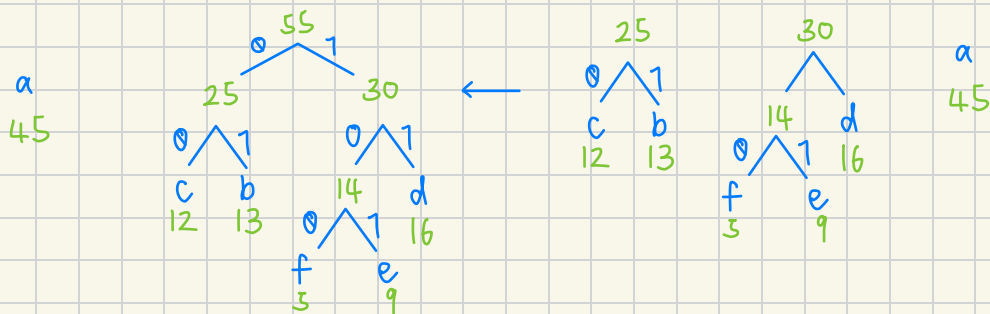
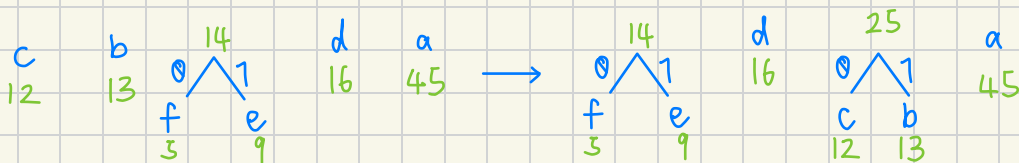
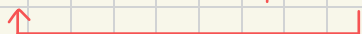
$\rightarrow O(n)$

Time Complexity: $O(n) + O(n \lg n) = O(n \lg n)$

Huffman Code

(a) f:5 e:9 c:12 b:13 d:16 a:45

每次都贪心的取频率最少的2个 → 合并 → 放入樹中



* C 是字元集合

Huffman Coding (C)

$n \leftarrow C.length()$ // 取得字元數量

$Q \leftarrow C$ // 把 C 中所有字元當葉節點, 建一個以頻率 f 為 key 的 min priority-queue, $O(n)$

for $i \leftarrow 1 \sim n-1$

create new node z

$x \leftarrow \text{ExtractMin}(Q)$ // $O(\lg n)$

$y \leftarrow \text{ExtractMin}(Q)$ // $O(\lg n)$

$z.left \leftarrow x$

$z.right \leftarrow y$

$f[z] \leftarrow f[x] + f[y]$ // 新 node 的頻率是 2 子節點頻率之和

Insert(Q, z) // 把新的 node z 插回 Q

return $\text{ExtractMin}(Q)$ // loop 結束後, Q 只剩下一個 node, 即整棵 Huffman Tree 的 root

Time Complexity: $\underbrace{O(n)}_{\text{建 } Q \text{ 所需時間}} + \underbrace{(n-1) \times O(\lg n)}_{\text{調整 min priority queue 所花時間}} = O(n \lg n)$